bluTruth — Bluetooth Stack Diagnostic Platform

Knowledge Base, Architecture, and Developer Reference

> **Project:** bluTruth (intentional spelling) **Mission:** The first tool that correlates all Bluetooth diagnostic streams simultaneously — HCI events, D-Bus signals, kernel traces, and daemon logs — into a unified, timestamped, queryable timeline. **Status:** Discovery Phase → Architecture Definition **Date:** 2026-02-23

Table of Contents

1. Why bluTruth Exists
2. The Full Linux Bluetooth Stack
    - Layer 0: Hardware
    - Layer 1: Kernel Driver — btusb.ko / hci_uart.ko
    - Layer 2: Core Kernel Module — bluetooth.ko
    - Layer 3: bluetoothd (BlueZ Daemon)
    - Layer 4: Audio Subsystem — PipeWire / PulseAudio
    - Layer 5: Applications
3. Where Problems Actually Live
4. BlueZ History
5. Diagnostic Streams Available
6. bluTruth Architecture
    - Core Principle: Unified Timeline
    - Component Map
    - Data Collection Layer
    - Correlation Engine
    - Storage Layer
    - Interface Layer — TUI and HTTP
7. Feature Specifications
    - Adapter and Pairing DB Management
    - Stack Restart Controls
    - D-Bus Introspection Monitor
    - Protocol Flow View

- Kernel Module Probe

8. Tools Built So Far
9. Blind Spots — Stack Layers With No Visibility
10. Commands and Programs Reference
11. Links for More Info

Why bluTruth Exists

When a Bluetooth connection fails on Linux the failure mode is almost always silent or cryptic. You don't get a clear error message. You get a device that won't pair, audio that drops, a peripheral that reconnects then immediately disconnects, or a daemon that silently transitions into a broken state with no indication of why.

Diagnosing this requires correlating at minimum four separate data streams:

- **HCI events** from `btmon` — what the hardware and kernel protocol stack are doing
- **D-Bus signals** from `bluetoothd` — what the daemon thinks is happening and what decisions it's making
- **Kernel trace events** from `debugfs` — internal kernel BT subsystem state
- **Daemon debug logs** from `bluetoothd -d` — why the daemon made the choices it did

These four streams exist in isolation. No existing tool combines them. When debugging you're manually correlating timestamps across four terminal windows. This is what makes Bluetooth on Linux hard to diagnose — not that the information doesn't exist, but that it's fragmented and uncorrelated.

bluTruth is the tool that doesn't exist yet. It ingests all four streams, timestamps every event to microsecond precision, correlates them into a unified timeline, and lets you query, filter, and replay what happened during a failure.

The Full Linux Bluetooth Stack

```
Your App (Spotify, etc.)
        |
PipeWire / PulseAudio
        |
BlueZ profile plugins (A2DP, HFP, HID, RFCOMM...)
        |
bluetoothd  <-->  D-Bus  <-->  desktop/CLI tools
        |
mgmt API (netlink socket to kernel)
        |
```

```
core  bluetooth.ko
        |
btusb.ko / hci_uart.ko
        |
hardware (USB dongle / PCIe card / UART chip)
```

Each boundary is a point where things can silently fail. Understanding what lives at each layer is the prerequisite for knowing which layer is responsible for a given failure.

Layer 0: Hardware

What it is

The physical Bluetooth controller — either a USB dongle, an integrated PCIe/USB chip on a laptop motherboard, or a UART-connected chip on embedded hardware. This contains its own microcontroller running its own firmware, completely separate from the Linux kernel.

What the hardware actually does

The controller handles everything at the RF level and below the HCI boundary. It manages the radio, frequency hopping, timing, and low-level packet framing. It implements the Link Controller and Link Manager (for Classic BT) or the Link Layer (for BLE). From the host stack's perspective, the controller is a black box that accepts HCI commands and returns HCI events and data.

Firmware

This is a critical and often overlooked source of bugs. The controller has its own firmware, completely separate from Linux kernel modules. Common chip families and their firmware situation:

Intel (most modern laptops) — firmware loaded at driver init from `/lib/firmware/intel/`. Intel actively maintains these and ships updates. Generally the best-supported on Linux.

Realtek (very common in cheap USB dongles) — firmware from `/lib/firmware/rtl_bt/`. Quality varies enormously by chip generation. RTL8761B and RTL8852A have reasonable support. Older chips have poor or unmaintained firmware.

Broadcom/Cypress (Macs, some laptops) — firmware from `/lib/firmware/brcm/`. macOS-first, Linux support is secondary. bcm43xx chips are common here.

MediaTek — increasingly common in newer hardware. Firmware from `/lib/firmware/mediatek/`. Support has improved significantly in recent kernels (5.15+).

Marvell — older laptops and embedded. `/lib/firmware/mrvl/`. Relatively stable but not updated often.

The bcdDevice field

When you read `/sys/bus/usb/devices/<dev>/bcdDevice`, you're reading the firmware revision that the hardware reports in its USB descriptor. This is the version burned into the controller's flash. It is distinct from the firmware file loaded by the driver — some controllers load new firmware at every driver init and update their reported version, others report the factory-flashed version regardless.

Tracking this field over time (which `bt_probe` does) lets you detect silent firmware changes after system updates.

What has no visibility

The controller's internal state is almost entirely opaque. You can't see its link manager state, its scheduling decisions, its RF-level retry counters, or its internal error rates. You see only what it chooses to tell you through HCI events. If the controller firmware has a bug that causes it to silently drop packets without sending an error event, you will not know from the host side.

Layer 1: Kernel Driver

btusb.ko

The USB transport driver for Bluetooth. Its job is narrow and specific: read USB endpoints and turn raw bytes into HCI packets, and turn HCI packets into USB writes. It understands the USB class structure (class 0xe0, subclass 0x01, protocol 0x01 identifies a BT controller) and knows how to load firmware for specific vendor/product ID combinations.

Key parameters exposed via `/sys/module/btusb/parameters/`:

`reset` — force a hardware reset on probe. Useful when a controller gets into a bad state after suspend/resume.

`enable_autosuspend` — whether to allow the USB controller to enter autosuspend when idle. Disabling this (set to 0) can fix intermittent failures on systems where the controller does not wake cleanly from autosuspend.

What btusb does at init:

1. Recognizes the USB device as a BT controller
2. Loads vendor-specific firmware if needed (dispatches to btrtl, btbcm, btintel, etc.)
3. Registers an HCI device with the core `bluetooth` module
4. Sets up three USB endpoints: interrupt (for HCI events), bulk-in (for ACL data from controller), bulk-out (for ACL data to controller)
5. SCO audio uses isochronous endpoints on some hardware, synchronous on others — this is a persistent source of audio quality issues

Vendor sub-drivers

`btrtl.ko` — Realtek firmware loading logic. `btbcm.ko` — Broadcom/Cypress firmware loading. `btintel.ko` — Intel firmware loading. Most sophisticated — handles multiple boot stages and patches. `btmtk.ko` — MediaTek.

These are not independent. `btusb` loads them as helpers. They contain vendor-specific knowledge of how to talk to each chip family during initialization.

hci_uart.ko

Same role as `btusb` but for UART-connected controllers. Common on Raspberry Pi and embedded hardware. Uses different flow control and transport framing (H4, BCSP, LL, or Three-wire UART protocols).

What you can see

- Module load/unload via `lsmod`
- Runtime parameters via `/sys/module/btusb/parameters/`
- Firmware loading events in `dmesg`
- USB device attributes in `/sys/bus/usb/devices/`
- Driver binding in `/sys/bus/usb/devices/<iface>/driver`

What you can change at runtime

```bash
# Disable autosuspend
echo 0 > /sys/module/btusb/parameters/enable_autosuspend

# Force reset on next probe - requires rmmod/modprobe cycle
rmmod btusb && modprobe btusb reset=1
```

Layer 2: Core Kernel Module

bluetooth.ko — The Foundation

This module is the operating system's definition of what Bluetooth is from a software perspective. It does not touch hardware. What it does is implement the core protocol stack and expose the kernel socket interface that everything else uses.

Every other BT module depends on it. `btusb` needs it. `rfcomm` needs it. `bnep` needs it. If you could unload it (you almost never can because refcnt is always high), the entire stack collapses.

What lives inside it

HCI layer — the Host Controller Interface implementation. Manages the HCI command queue, handles flow control between host and controller, dispatches incoming events to the right protocol handler, maintains the device registry, and enforces the HCI spec's command/event sequencing rules. When `btmon` shows you HCI commands and events, it is tapping into this layer via a special monitoring socket type (`HCI_MONITOR`).

L2CAP — Logical Link Control and Adaptation Protocol. The multiplexing layer that sits above HCI. Lets multiple protocols share a single physical connection by assigning channel IDs (CIDs). Classic L2CAP and LE L2CAP (called L2CAP CoC — Connection-oriented Channels — in BLE) are both implemented here. Most of the handshake traffic you see in btmon is L2CAP signalling: configuration requests, connection requests, information requests, MTU negotiation, and feature exchanges.

The mgmt API — the modern userspace control interface. Implemented as a netlink socket that `bluetoothd` connects to. Replaced direct HCI socket access for management operations in BlueZ 5. Handles adapter power, discovery, pairing, security policies, and configuration. The `btmgmt` CLI tool talks directly to this interface, bypassing bluetoothd entirely.

AF_BLUETOOTH socket family — registers a new address family with the kernel so userspace can open Bluetooth sockets. Socket protocols within this family:

Protocol constant	Value	Purpose
BTPROTO_L2CAP	0	Direct L2CAP sockets
BTPROTO_HCI	1	Raw HCI access / monitoring
BTPROTO_SCO	2	Synchronous audio connections
BTPROTO_RFCOMM	3	Serial port emulation
BTPROTO_BNEP	4	Bluetooth network encapsulation
BTPROTO_CMTP	5	CAPI message transport
BTPROTO_HIDP	6	HID protocol

SMP (Security Manager Protocol) for BLE — pairing, key distribution, and encryption setup for LE connections. The key exchange that produces LTK (Long Term Key), IRK (Identity Resolving Key), and CSRK (Connection Signature Resolving Key) happens at this layer.

Key parameters

```bash
```

```
ls /sys/module/bluetooth/parameters/
```

`disable_esco` — eSCO is Enhanced Synchronous Connection, used for higher quality audio codecs (mSBC, CVSD) in HFP. Disabling forces fallback to basic SCO. Set to 1 if you are seeing HFP audio connection failures on headsets.

`enable_le` — enables BLE functionality. Almost always 1. Setting to 0 disables all BLE at the kernel level.

`enable_hs` — Alternate MAC/PHY high-speed extension over 802.11. Rarely used. Often compiled out entirely.

`hci_to` — HCI command timeout in seconds. Default 10. If you are seeing `Command Status: Timeout` in btmon, look here.

`enable_mgmt` — whether the management interface is exposed to userspace. If 0, bluetoothd cannot function. Should always be 1.

Changing parameters at runtime

Parameters can be written via sysfs but most only take effect when the adapter is re-initialized:

```bash
echo 1 > /sys/module/bluetooth/parameters/disable_esco
# Bounce the adapter to apply
hciconfig hci0 down && hciconfig hci0 up
```

To change parameters that only affect module init, you need a full reload:

```bash
systemctl stop bluetooth
rmmod bnep rfcomm btusb bluetooth
modprobe bluetooth disable_esco=1
modprobe btusb
systemctl start bluetooth
```

Security policy decisions made here

The core module is where the kernel decides whether to accept a connection from an unknown device, what security level to require before allowing a profile connection, whether to accept a downgrade from Secure Simple Pairing to legacy PIN pairing, and how to handle a LE device requesting downgrade from Secure Connections to legacy LE pairing.

These decisions are reflected in btmon as `Security Manager Protocol` events and are critical context for understanding suspicious activity. An attacker attempting a downgrade attack (KNOB attack, BIAS attack) produces specific patterns here that bluTruth's anomaly detection should flag.

Layer 3: bluetoothd

Overview

The userspace daemon that sits between the kernel and applications. Manages pairing state, profiles, device databases, and the D-Bus API that desktop environments and applications use. Everything that is not raw protocol implementation lives here.

This is historically where most Linux Bluetooth reliability problems originate.

What it owns

Device and pairing database — stored in `/var/lib/bluetooth/<adapter_address>/`. Flat-file structure:

```
/var/lib/bluetooth/
  AA:BB:CC:DD:EE:FF/          <- adapter address
    cache/
      XX:XX:XX:XX:XX:XX       <- cached device info (name, class, manufacturer)
    XX:XX:XX:XX:XX:XX/        <- paired device directory
      info                    <- pairing info, keys, services
    settings                  <- adapter-level settings
```

The `info` file for a paired device contains several INI-format sections. The `[General]` section holds name, class, appearance, and supported/blocked profiles. The `[LinkKey]` section holds the Classic BT link key for BR/EDR. The `[LongTermKey]` section holds the BLE LTK plus authentication and encryption metadata. The `[LocalSignatureKey]` and `[RemoteSignatureKey]` sections hold the CSRK for signed writes. The `[IdentityResolvingKey]` section holds the IRK for private address resolution. The `[ConnectionParameters]` section holds BLE connection interval, latency, and supervision timeout.

This database is a frequent source of failures. Stale entries, corrupted keys, or mismatched security parameters between the stored entry and what the device now expects all produce silent failures where the device connects at HCI level but immediately fails at a higher layer.

Profile manager — registers profiles with the kernel and negotiates which profiles activate when a device connects. Profiles are implemented as plugins loaded at daemon startup. If a profile plugin fails to initialize (common after BlueZ version changes), the connection succeeds at HCI level but the profile is unavailable with no clear error surfaced to the user.

Agent system — handles pairing UI. When a device initiates pairing, bluetoothd asks a registered agent to provide PIN/passkey/confirmation. If no agent is registered (headless systems, broken desktop integration), pairing silently fails. The agent interface is exposed over D-Bus at `org.bluez.Agent1`.

Auto-connect logic — attempts reconnection to known devices. Has historically had race conditions where reconnect is attempted before the kernel adapter is fully initialized after resume from suspend, poisoning

connection state for that session.

Policy plugin — implements connection policy decisions: which devices are trusted, which profiles are allowed, and what security modes to enforce.

D-Bus API surface

bluetoothd exposes its entire state through D-Bus under the `org.bluez` service name on the system bus. The object hierarchy:

```
/org/bluez/
  hci0                    <- Adapter1 interface
    dev_XX_XX_XX_XX_XX_XX        <- Device1 interface
      player0                 <- MediaPlayer1 (if connected)
      sep0                    <- MediaEndpoint1 (A2DP)
  hci1                    <- second adapter if present
```

Key interfaces:

`org.bluez.Adapter1` — adapter management. Properties: `Powered`, `Discoverable`, `Pairable`, `Discovering`, `Address`, `Name`, `Class`, `UUIDs`. Methods: `StartDiscovery`, `StopDiscovery`, `RemoveDevice`, `SetDiscoveryFilter`.

`org.bluez.Device1` — per-device state. Properties: `Connected`, `Paired`, `Trusted`, `Blocked`, `RSSI`, `TxPower`, `ManufacturerData`, `ServiceData`, `UUIDs`, `ServicesResolved`. Methods: `Connect`, `Disconnect`, `Pair`, `CancelPairing`.

`org.bluez.MediaEndpoint1` — A2DP codec endpoint negotiation. This is where codec selection happens and where A2DP failures often manifest in ways that are not reflected in HCI events.

`org.bluez.ProfileManager1` — register/unregister custom profiles.

`org.bluez.AgentManager1` — register/unregister pairing agents.

`org.bluez.GattManager1` — register/unregister GATT applications.

Monitoring bluetoothd's internal state

The standard tools give you the D-Bus surface. To see what is happening inside:

```bash
```

```
# Watch all D-Bus signals and property changes from bluetoothd in real time
dbus-monitor --system "sender=org.bluez"

# Full object tree — everything bluetoothd knows about
busctl tree org.bluez

# Deep property dump for a specific device
busctl introspect org.bluez /org/bluez/hci0/dev_XX_XX_XX_XX_XX_XX

# Watch property changes live
busctl monitor org.bluez

# Debug logging — most revealing but overwhelming
systemctl stop bluetooth
bluetoothd -n -d 2>&1 | tee /tmp/bt_debug.log
```

Restarting bluetoothd

```bash
systemctl restart bluetooth
# With pause for cleaner state reset
systemctl stop bluetooth && sleep 2 && systemctl start bluetooth
# With adapter reinit
systemctl stop bluetooth
hciconfig hci0 down && hciconfig hci0 up
systemctl start bluetooth
```

Wiping and reloading the pairing database

This resolves a large class of silent "device won't connect or pair" failures:

```bash
# Nuclear — wipes all pairing info for all devices on this adapter
systemctl stop bluetooth
rm -rf /var/lib/bluetooth/AA:BB:CC:DD:EE:FF/*
systemctl start bluetooth

# Surgical — wipe one device
systemctl stop bluetooth
rm -rf "/var/lib/bluetooth/AA:BB:CC:DD:EE:FF/XX:XX:XX:XX:XX:XX"
systemctl start bluetooth
```

After wiping, the device must be re-paired and the pairing must also be forgotten on the device side. Both ends must agree they are starting from scratch.

Layer 4: Audio Subsystem

The boundary problem

Bluetooth audio involves a handoff between bluetoothd (which owns the BT connection and profile negotiation) and the audio server (PipeWire or PulseAudio, which owns the audio pipeline). This handoff is a historically messy boundary and a major source of failures.

A2DP (Advanced Audio Distribution Profile)

Used for high-quality stereo audio output to headphones and speakers. The codec negotiation happens at the BlueZ level — bluetoothd and the audio server jointly decide which codec to use. Common codecs:

SBC — mandatory baseline. Low quality but universally supported. Will always be the fallback.

AAC — high quality. Apple devices prefer this. Support in Linux has improved significantly in recent BlueZ versions.

aptX / aptX HD — Qualcomm proprietary. Requires license. Some distributions ship without it.

LDAC — Sony proprietary. High quality. Linux support via PipeWire 0.3.x and freeaptX.

LC3 — Bluetooth 5.2 (LE Audio). Very new, limited device support as of 2026.

Codec negotiation failures are a common A2DP problem. Two devices can both support AAC but disagree on specific parameters such as bitrate, channel count, or sampling frequency. If negotiation fails, the stack may silently fall back to SBC or fail entirely depending on the BlueZ version.

HFP (Hands-Free Profile)

Used for two-way audio (calls, voice assistants). Fundamentally different from A2DP — uses SCO/eSCO connections rather than ACL, and the audio path goes through the kernel's SCO socket layer rather than the normal audio stack.

HFP has two audio codec modes. Narrowband uses CVSD — legacy telephone quality. Wideband uses mSBC — much better quality and requires eSCO.

The `disable_esco` kernel parameter in `bluetooth.ko` directly affects whether wideband HFP works. If it is set to 1, all HFP falls back to narrowband.

The HFP audio path on Linux (especially with PipeWire) has been one of the most consistently broken areas. The SCO socket handoff between bluetoothd and the audio server has race conditions that cause dropped call audio on many systems.

PipeWire vs PulseAudio

PipeWire (the modern replacement) handles Bluetooth audio better than PulseAudio in most scenarios but brought its own bugs during the transition period from 2021 to 2023. On modern systems (PipeWire 0.3.50+) with recent BlueZ (5.65+), audio reliability is significantly better than it was.

The Bluetooth plugin in PipeWire (`libspa-bluez5`) handles both A2DP and HFP and is responsible for registering media endpoints with bluetoothd. If this plugin fails to load, Bluetooth audio will not work regardless of whether the BT connection itself is fine.

Layer 5: Applications

This is the outermost layer — the one users actually interact with. But unlike every layer below it, applications do not talk to the Bluetooth stack directly. They talk to two intermediaries: **bluetoothd** (via D-Bus) for everything except audio, and the **audio server** (PipeWire or PulseAudio) for audio output and input. Understanding this indirection is essential because failures that look like application problems are almost always failures in one of those intermediaries.

How Applications Talk to bluetoothd

All application interaction with the Bluetooth stack goes through the D-Bus system bus using the `org.bluez` service name. The complete API is defined in BlueZ's `doc/` directory in the source tree. The key interfaces are:

`org.bluez.Adapter1` — controls the physical adapter. Power on/off, start/stop discovery, set adapter properties (alias, discoverable, pairable, discoverable timeout). Every application that needs to scan for devices or change adapter state uses this.

`org.bluez.Device1` — represents a remote device. Connect, Disconnect, Pair, CancelPairing, ConnectProfile (by UUID), DisconnectProfile. Properties include Address, Name, Class, Appearance, RSSI, Connected, Paired, Trusted, Blocked, ServicesResolved, UUIDs. The `ServicesResolved` property is critical — service discovery only completes after `ServicesResolved: true`, and applications that try to use GATT or profiles before this point will fail silently.

`org.bluez.GattManager1` and **`org.bluez.GattApplication1`** — for apps that want to expose their own GATT services (peripheral role). An application registers an object implementing GattApplication1 with bluetoothd, which then advertises those services.

`org.bluez.GattService1`, **`org.bluez.GattCharacteristic1`**, **`org.bluez.GattDescriptor1`** — the object path hierarchy representing GATT on a connected device. Once `ServicesResolved` is true, bluetoothd populates the full GATT tree under the device's object path, and an application can call `ReadValue` and `WriteValue` directly on characteristic objects.

`org.bluez.ProfileManager1` — for registering custom RFCOMM profiles. Applications that want to open an RFCOMM serial port connection call `RegisterProfile` with a UUID, then receive incoming connections as Unix socket file descriptors passed over D-Bus.

org.bluez.AgentManager1 — for registering a pairing agent. If no agent is registered when a pairing requires user input (PIN, passkey, confirmation), the pairing will silently fail or use Just Works, which is a significant security downgrade. Desktop environments register agents automatically. In headless or embedded setups, you must register one manually or accept Just Works for everything.

org.bluez.NetworkServer1 / **org.bluez.Network1** — for BNEP networking (Bluetooth PAN). Applications wanting to use Bluetooth for IP networking use this.

org.bluez.InputDevice1 — for HID devices. Most input device handling is automatic once paired, but the interface is exposed for querying properties.

org.bluez.MediaControl1 / **org.bluez.MediaPlayer1** — for AVRCP media control. Media players that want to expose transport controls (play, pause, seek, track info) to connected BT devices register through this interface.

The D-Bus Permission Problem

The D-Bus system bus uses a policy file to control which users and processes can call which methods. The BlueZ D-Bus policy file is at `/etc/dbus-1/system.d/bluetooth.conf`. By default, only root and members of the `bluetooth` group can call most methods. Applications running as a regular user without group membership will receive `org.freedesktop.DBus.Error.AccessDenied` — and many BT applications fail silently when this happens rather than reporting the actual error clearly.

To verify permissions are the problem:

```bash
dbus-send --system --print-reply \
  --dest=org.bluez /org/bluez/hci0 \
  org.freedesktop.DBus.Properties.Get \
  string:org.bluez.Adapter1 string:Powered
```

If this returns a value, permissions are fine. If it returns an error, the calling user needs to be in the `bluetooth` group or the policy file needs adjustment.

How Applications Talk to Audio (PipeWire / PulseAudio)

BT audio applications never interact with BlueZ directly for audio data. The flow is:

1. Application opens an audio stream via PipeWire (modern) or PulseAudio (legacy) using the standard ALSA / PipeWire / PulseAudio APIs.
2. The audio server has a Bluetooth plugin (`spa-bluez5` in PipeWire, `module-bluetooth-discover` in PulseAudio) that talks to bluetoothd to establish the A2DP or HFP audio transport.
3. The plugin negotiates codec and opens the BlueZ media transport.
4. Audio data flows: application → audio server → BT plugin → BlueZ media transport → kernel → HCI → controller → air.

The application sees none of this. It sees a PipeWire audio sink named something like "Headphones - A2DP". If the audio sink doesn't appear, the failure is either in the BT connection layer (bluetoothd didn't connect A2DP profile), the codec negotiation layer (A2DP profile connected but couldn't agree on codec), or the media transport layer (profile connected but the audio server's plugin failed to open it).

Common Application-Level Failure Modes

`ServicesResolved` **race condition** — the most common one. An application connects to a device, the `Connected` property goes true, and the application immediately tries to access GATT characteristics. But service discovery hasn't completed yet — `ServicesResolved` is still false. The read fails, often silently. Fix: subscribe to `PropertiesChanged` and wait for `ServicesResolved: true` before accessing GATT.

No agent registered — pairing initiated programmatically will silently fail if no agent is registered to handle the user input step. The pairing process reaches the step requiring confirmation and gets no response. Fix: register an agent or ensure one is running (a desktop environment's BT applet handles this on desktop systems).

Wrong UUID format — BlueZ requires full 128-bit UUIDs in lowercase in many contexts. Applications that use short UUIDs (16-bit or 32-bit) in the wrong place will fail with cryptic errors. The correct expanded form of, say, `0x1234` is `00001234-0000-1000-8000-00805f9b34fb`.

Stale device object — a device was removed from bluetoothd's device registry but the application still holds a D-Bus object reference to it. Calls on the stale reference return `org.bluez.Error.DoesNotExist`. Fix: subscribe to `InterfacesRemoved` signals and invalidate references.

D-Bus timeout on slow operations — GATT characteristic reads on slow devices can take longer than the default D-Bus method call timeout. Applications that don't set an extended timeout will see a timeout error even though the operation would eventually succeed.

Audio sink appearing before transport is ready — PipeWire may announce the BT audio device as available before the audio transport is fully established. Applications that immediately try to open it get a stream that opens but produces no audio or immediately drops.

Diagnostic Commands for This Layer

```bash
```

```
# Watch all D-Bus traffic from bluetoothd in real time
dbus-monitor --system "sender=org.bluez"

# Check what bluetoothd currently knows about a device
busctl introspect org.bluez /org/bluez/hci0/dev_AA_BB_CC_DD_EE_FF

# Get all properties of a device
busctl call org.bluez /org/bluez/hci0/dev_AA_BB_CC_DD_EE_FF \
  org.freedesktop.DBus.Properties GetAll s org.bluez.Device1

# List all GATT services and characteristics resolved for a device
busctl tree org.bluez

# Check if an application can access bluetoothd at all
dbus-send --system --print-reply \
  --dest=org.bluez / org.freedesktop.DBus.ObjectManager.GetManagedObjects

# Watch PipeWire BT transport state
pw-dump | python3 -c "import sys,json; [print(n) for n in json.load(sys.stdin) if 'bluez' in str(n).lower()]"

# PulseAudio BT device list
pactl list cards | grep -A20 bluez
```

What This Layer Looks Like in bluTruth

The D-Bus Monitor tab in bluTruth is specifically designed to give visibility into this layer. Every property change on every `/org/bluez/*` object path appears with a timestamp alongside the HCI stream. This means you can see:

- Exactly when `Connected: false → true` relative to the HCI `Connection Complete` event
- When `ServicesResolved: false → true` relative to the GATT attribute protocol exchanges in the HCI stream
- When a media transport state changes from `idle → pending → active` relative to the A2DP profile negotiation in btmon

Without this correlation, the application layer is a black box. With it, you can pinpoint exactly which layer is responsible for a failure — because you'll see either the HCI event that should have triggered a state change not being followed by the D-Bus property update (bluetoothd bug), or the property update happening but the application not reacting to it (application bug).

Where Problems Actually Live

A realistic breakdown based on observed failure frequency:

Layer	Problem Source	Frequency	Typical Symptom
Hardware firmware	Controller bugs, Realtek/cheap dongles	High	Random disconnects, won't init
btusb.ko	Autosuspend wake failures	Medium	Fails after resume from suspend
bluetooth.ko	Rare — very stable	Low	Usually a kernel version regression
bluetoothd state machine	Race conditions, reconnect bugs	High	Won't reconnect, stuck state
Pairing database	Stale/corrupt entries	High	Pairs then immediately drops
Agent system	No agent registered	Medium	Silent pairing failure
Profile negotiation	BlueZ and audio server handoff	High	Connected but no audio
Codec negotiation	A2DP parameter mismatch	Medium	Falls back to SBC unexpectedly
Audio server (PipeWire)	Plugin failures, SCO handoff	High	Audio drops, HFP broken
Application	D-Bus perms, wrong UUID	Low	App-specific failure

Device vs host: The device (headphones, phone, peripheral) is more often the proximate cause because consumer firmware deviates from spec in ways that strict stacks reject. But the Linux stack's lack of visibility into why it rejected something makes it look like a Linux problem even when the device is at fault. This is the asymmetry that better tooling can resolve — if you can see that the Linux stack rejected a connection because the device proposed an illegal security parameter, you can stop blaming Linux.

BlueZ History

2001 — BlueZ started by Qualcomm engineers Maxim Krasnyansky and Marcel Holtmann. Goal: a working Bluetooth stack in the Linux kernel.

2001–2006 — Initial kernel integration. Separate daemons: `hcid`, `sdpd`, `rfcommd`. Basic profiles: HSP, HFP, A2DP, RFCOMM, BNEP.

2006–2012 — BlueZ 4.x era. Stable but architecturally fragmented. BLE support bolted on as the spec evolved.

2012 — BlueZ 5.0 released. Near-complete rewrite. All daemons merged into single `bluetoothd`. Complete D-Bus API redesign. mgmt API replaces direct HCI socket access. Proper BLE support from the ground up. Broke

essentially everything built on BlueZ 4.x.

2012–2016 — Transitional chaos. Distributions shipped BlueZ 5 before the ecosystem adapted. PulseAudio, desktop environments, and third-party tools all broke simultaneously. This is when the "Linux Bluetooth is garbage" reputation solidified. It was largely deserved at the time.

2016–2020 — Stabilization. Audio pipelines improved. BLE peripherals became reliable. HFP and complex reconnection scenarios remained fragile.

2020–present — PipeWire transition. New audio architecture resolved many longstanding HFP and A2DP issues. Bluetooth 5.x features being added incrementally. Marcel Holtmann remains primary maintainer.

Diagnostic Streams Available

These are all the data sources bluTruth can tap.

Stream 1: HCI Monitor (btmon)

```bash
btmon
btmon -w /tmp/capture.btsnoop   # save to Wireshark-compatible file
```

Opens an `HCI_MONITOR` socket in the kernel. Captures every HCI command, event, and data packet in real time with kernel timestamps. The single most informative stream — everything at and below L2CAP is visible here. Important constraint: only one consumer can open the monitor socket at a time. bluTruth needs to run a broadcaster/fan-out so multiple consumers (TUI tabs, HTTP stream, disk capture) all share one connection.

Stream 2: D-Bus signals from bluetoothd

```bash
dbus-monitor --system "sender=org.bluez"
busctl monitor org.bluez
```

Property changes on all adapters and devices. `InterfacesAdded` and `InterfacesRemoved` when devices appear and disappear. `PropertiesChanged` for `Connected`, `Paired`, `RSSI`, and anything else bluetoothd tracks. This is the window into bluetoothd's internal state machine — what it currently believes about every device.

Stream 3: bluetoothd debug log

```bash
bluetoothd -n -d 2>&1
```

Verbose output includes profile negotiation decisions, agent interactions, SDP service record resolution, internal state transitions, and the reasons it rejected connections or refused pairings. Not available via systemd journal without restarting the daemon in foreground mode.

Stream 4: Kernel trace events

```bash
echo 1 > /sys/kernel/debug/tracing/events/bluetooth/enable
cat /sys/kernel/debug/tracing/trace_pipe
```

Below the HCI level — internal kernel BT subsystem events that do not surface in btmon. Useful for diagnosing kernel-level failures in the mgmt API layer. Requires root and debugfs mounted.

Stream 5: dmesg

Firmware loading events, driver init errors, hardware detection. One-time events at boot or device plug-in rather than a real-time stream, but essential for startup and resume failure diagnosis.

Stream 6: /proc/net/bluetooth

```bash
ls /proc/net/bluetooth/
cat /proc/net/bluetooth/l2cap
```

Open socket state per protocol. Snapshot — not a stream. Shows what connections are actually open at the socket level right now.

Stream 7: sysfs adapter state

```bash
cat /sys/class/bluetooth/hci0/address
cat /sys/class/bluetooth/hci0/features
cat /sys/kernel/debug/bluetooth/hci0/*
```

Adapter-level state. Snapshot. debugfs gives the most detail but requires root.

The Gap (What bluTruth Fills)

No existing tool correlates these streams with a unified timestamp. When a connection fails you see a `Disconnect Complete, reason=0x08 (timeout)` in btmon, a `Connected: false` signal on D-Bus, a decision made in the bluetoothd log 200ms earlier, and a kernel internal state transition 50ms before that. These four events are causally related but timestamped by different clocks in different log files. bluTruth's job is to ingest all of them,

normalize the timestamps, and present them as a single coherent timeline so you can see cause and effect rather than disconnected symptoms.

---

bluTruth Architecture

Core Principle: Unified Timeline

Every event from every source gets a normalized monotonic timestamp (using `CLOCK_MONOTONIC_RAW` or `CLOCK_BOOTTIME`), a source tag (HCI / DBUS / DAEMON / KERNEL / SYSFS), a severity classification (INFO / WARN / ERROR / SUSPICIOUS), a protocol stage tag (DISCOVERY / CONNECTION / HANDSHAKE / DATA / AUDIO / TEARDOWN), and a device address if relevant.

Events are stored in SQLite and streamed to both the TUI and HTTP interface in real time.

Component Map

```
+-------------------------------------------------------------+
|                    COLLECTION LAYER                         |
|                                                             |
| +----------+  +----------+  +----------+  +--------+       |
| | btmon    | | D-Bus    | |bluetoothd| | kernel |        |
| | listener | | monitor  | | log pipe | | tracer |        |
| +----+-----+  +----+-----+  +-----+----+  +---+----+       |
|      +---------------+-------------+-----------+            |
|                      |                          |
|              +------v------+                    |
|              | Normalizer | (timestamp, classify)          |
|              +------+------+                    |
+---------------+-------+--------------------------------+
                |
          +-----------v-----------+
          |  CORRELATION ENGINE   |
          | - timeline assembly   |
          | - event linking       |
          | - anomaly detection   |
          +-----------+-----------+
                      |
        +---------------+---------------+
        |             |               |
  +------v------+ +-----v------+ +------v-----+
  | SQLite DB   | | TUI (Rust/ | | HTTP API   |
  | (timeline)  | | Textual)   | | + Web UI   |
  +-------------+ +------------+ +------------+
```

```
        |              |
+------v------+ +------v------+
| bttui.py    | | blutruth.js |
| (existing)  | |  React SPA  |
+-------------+ +-------------+
```

Data Collection Layer

HCI Broadcaster

The btmon broadcaster singleton already built in `bttui.py`. Extend to output structured JSON events (not just raw text lines), parse HCI opcodes into human-readable form with protocol stage tags, emit parsed events on a unix socket for the HTTP backend to consume, and save btsnoop capture files alongside the SQLite timeline.

D-Bus Monitor (new — core of bluTruth)

Python implementation using `dbus-python`:

```python
```

```python
class DBusMonitor:
    def __init__(self, event_queue):
        self.bus = dbus.SystemBus()
        self.queue = event_queue

        # Watch all PropertiesChanged signals under /org/bluez
        self.bus.add_signal_receiver(
            self._on_properties_changed,
            dbus_interface="org.freedesktop.DBus.Properties",
            signal_name="PropertiesChanged",
            path_namespace="/org/bluez",
            sender_keyword="sender",
            path_keyword="path",
        )

        # Watch device appear/disappear
        self.bus.add_signal_receiver(
            self._on_interfaces_added,
            dbus_interface="org.freedesktop.DBus.ObjectManager",
            signal_name="InterfacesAdded",
        )
        self.bus.add_signal_receiver(
            self._on_interfaces_removed,
            dbus_interface="org.freedesktop.DBus.ObjectManager",
            signal_name="InterfacesRemoved",
        )

    def _on_properties_changed(self, interface, changed, invalidated,
                               sender=None, path=None):
        event = {
            "ts": time.time(),
            "source": "DBUS",
            "path": path,
            "interface": interface,
            "changed": {str(k): str(v) for k, v in changed.items()},
        }
        self.queue.put(event)
```

Rust implementation using `zbus`:

```rust
```

```
use zbus::{Connection, MessageStream};
use futures_util::stream::StreamExt;

let connection = Connection::system().await?;
let stream = MessageStream::from(&connection);
// Filter for org.bluez signals and dispatch to correlation engine
```

Adapter and DB snapshot (bt_probe.py / bt_probe.rs — existing)

Already built. Integrate as the static probe that runs at startup and on demand. Probe results appear in the timeline as SYSFS-tagged snapshot events.

bluetoothd debug log capture

For maximum visibility, bluTruth should optionally manage bluetoothd itself — launch it as a subprocess with the `-d` flag and capture stdout/stderr directly. For systems where the daemon runs under systemd, capture via `journalctl -f -u bluetooth`.

Kernel trace collector

```bash
mount -t debugfs none /sys/kernel/debug
echo 1 > /sys/kernel/debug/tracing/events/bluetooth/enable
cat /sys/kernel/debug/tracing/trace_pipe   # blocking read, line by line
```

Correlation Engine

The core algorithmic component.

Timestamp normalization — all sources use different clock sources. btmon uses `CLOCK_REALTIME`. D-Bus signals have system time. Kernel traces use monotonic clock. The correlation engine normalizes all to a common reference using boot time as the anchor.

Event linking — when a D-Bus `Connected: false` appears, find the btmon `Disconnect Complete` event within ±50ms and link them as the same logical event with different perspectives. A single connection failure should appear as one grouped event with four sub-perspectives rather than four unrelated log lines.

Device address resolution — BLE devices use random addresses that rotate. The IRK stored in the pairing database resolves random addresses to known device identities. The correlation engine should resolve addresses so events from the same device are grouped regardless of address rotation.

Anomaly detection — pattern match against known failure signatures including KNOB attack pattern (encryption key length negotiation to 1 byte), BIAS attack pattern (authentication procedure bypass), repeated

pairing failures from same address, unexpected role switches, profile connection attempts on non-paired devices, eSCO downgrade to SCO, and repeated HFP codec negotiation failures.

State tracking — maintain a live model of what bluetoothd believes about each adapter and device. When an event arrives, record the before/after state transition. This enables replay: "what state was the system in when this failure occurred?"

Storage Layer

SQLite — single file, zero config, embeds in the process. Fast enough for BT event rates (100–500 events/sec peak during connection establishment).

Schema overview:

```sql
```

```sql
-- Every event from every source
CREATE TABLE events (
    id          INTEGER PRIMARY KEY,
    ts_mono_us  INTEGER NOT NULL,   -- microseconds since boot
    ts_wall     TEXT    NOT NULL,   -- ISO8601 wall clock
    source      TEXT    NOT NULL,   -- HCI|DBUS|DAEMON|KERNEL|SYSFS
    severity    TEXT    NOT NULL,   -- INFO|WARN|ERROR|SUSPICIOUS
    stage       TEXT,               -- DISCOVERY|CONNECTION|HANDSHAKE|DATA|AUDIO|TEARDOWN
    device_addr TEXT,               -- normalized BD address or null
    event_type  TEXT    NOT NULL,   -- HCI_CMD|HCI_EVT|DBUS_PROP|DBUS_SIG|...
    summary     TEXT    NOT NULL,   -- human-readable one-liner
    raw_json    TEXT    NOT NULL    -- full parsed event data
);

-- Device identity across address changes
CREATE TABLE devices (
    id            INTEGER PRIMARY KEY,
    canonical_addr TEXT UNIQUE,     -- stable address or IRK-resolved identity
    known_addrs   TEXT,             -- JSON array of seen addresses
    name          TEXT,
    class         TEXT,
    manufacturer  TEXT,
    first_seen    TEXT,
    last_seen     TEXT
);

-- Correlated event groups (one logical event = many source perspectives)
CREATE TABLE event_groups (
    group_id    INTEGER,
    event_id    INTEGER REFERENCES events(id),
    role        TEXT    -- PRIMARY|CORRELATED|CONTEXT
);

-- Adapter/pairing DB snapshots (from bt_probe)
CREATE TABLE snapshots (
    id          INTEGER PRIMARY KEY,
    captured_at TEXT,
    probe_json  TEXT
);

-- Named captures / sessions
CREATE TABLE sessions (
    id          INTEGER PRIMARY KEY,
```

```
    name        TEXT,
    started_at  TEXT,
    ended_at    TEXT,
    notes       TEXT
);
```

Retention policy — events are lightweight (under 1KB each). 24 hours of active BT use produces 50–500K events, roughly 100MB uncompressed. SQLite with WAL mode handles this comfortably. Implement rolling cleanup: keep last N days or N GB, configurable.

Interface Layer

TUI (Python Textual / Rust Ratatui)

Extend existing `bttui.py`. New tabs needed:

Timeline Tab — the centerpiece. Real-time scrolling event stream from all sources simultaneously. Color-coded by source and severity. Click an event to see its correlated group. Filter by device, source, stage, severity.

D-Bus Live Tab — real-time property changes from bluetoothd. Object tree on the left, property diff stream on the right. Shows exactly what bluetoothd knows about each device at this moment.

Device State Tab — per-device dashboard. For each known device: connection state, signal strength over time, profile status, pairing DB entry, recent events.

Controls Tab — the management interface: adapter power/reset/wipe, bluetoothd restart and debug relaunch, per-device pairing wipe and trust controls, full stack bounce operations.

Anomaly Tab — flagged events only. Pattern matches against known attack signatures and failure patterns.

HTTP + Web UI

A lightweight HTTP server embedded in the same process (or sidecar). Endpoints:

`GET /events?since=<ts>&source=<>&device=<>` — event stream as JSON or SSE for real-time push. `GET /state` — current adapter and device state snapshot. `GET /devices` — all known devices with current status. `POST /control` — management actions (restart, wipe, etc.). `GET /` — React SPA for browser access.

The web UI is valuable for remote debugging over SSH with port forwarding, attaching a second screen without a terminal multiplexer, sharing a live session, and long-running capture monitoring.

Technology choice:

Python backend: `aiohttp` for HTTP server plus `dbus-python` for D-Bus. Minimal dependencies, fast iteration.

Rust backend: `axum` plus `zbus` for D-Bus. Single binary, faster, better for deployment.

Recommended approach: Python for the collection and correlation engine during development (faster to iterate on the classification and correlation logic), Rust for the final production binary once the architecture is

proven.

Feature Specifications

Adapter and Pairing DB Management

Actions to implement:

```
Adapters
  - List all adapters with full state (from sysfs + hciconfig)
  - Power on/off
  - Reset (hciconfig down/up)
  - Full reinit (rmmod btusb, modprobe btusb)
  - View raw sysfs attributes

Pairing Database
  - List all paired devices with key metadata
  - View raw info file for any device
  - Wipe single device entry
  - Wipe all device entries for adapter
  - Export database (backup before wipe)
  - Import/restore database
  - Edit trust/block status
```

Reading the pairing DB is just file parsing — the `info` files are INI format. Writing requires stopping bluetoothd first since it holds these files open. The sequence for any DB modification: stop bluetoothd, make changes to `/var/lib/bluetooth/<adapter>/`, start bluetoothd, trigger re-scan. Some changes (Trusted, Blocked) can be made via D-Bus at runtime and bluetoothd will persist them. Key changes and device removal must go through `Adapter1.RemoveDevice` or require daemon stop.

Stack Restart Controls

```bash
```

```
# Full stack restart
systemctl stop bluetooth
hciconfig hci0 down
sleep 1
hciconfig hci0 up
systemctl start bluetooth

# Daemon-only restart
systemctl restart bluetooth

# btusb reload without daemon restart
systemctl stop bluetooth
rmmod btusb
sleep 1
modprobe btusb
systemctl start bluetooth

# Nuclear - full kernel stack reload
systemctl stop bluetooth
rmmod bnep rfcomm btusb bluetooth
sleep 2
modprobe bluetooth
modprobe btusb
systemctl start bluetooth
```

These should be one-click actions in the TUI and HTTP interface with confirmation dialogs. Optionally auto-capture a snapshot before and after each restart so you can diff what changed.

D-Bus Introspection Monitor

The most novel feature of bluTruth. A continuous, live view of bluetoothd's complete internal state.

Every `PropertiesChanged` signal on any `/org/bluez/*` path, every `InterfacesAdded` and `InterfacesRemoved` signal, and every method call and return are captured and displayed with millisecond timestamps alongside the HCI stream. Example display:

```
[14:32:01.234] DBUS  /org/bluez/hci0/dev_AA_BB_CC  PropertiesChanged  Device1
          Connected: false -> true

[14:32:01.891] DBUS  /org/bluez/hci0/dev_AA_BB_CC  PropertiesChanged  Device1
          ServicesResolved: false -> true

[14:32:02.103] DBUS  /org/bluez/hci0/dev_AA_BB_CC  PropertiesChanged  MediaTransport1
          State: idle -> pending
```

This tells you exactly when bluetoothd considers a device connected, when service resolution completes, and when the audio transport becomes active — all correlated with the HCI events happening simultaneously in the adjacent column.

Protocol Flow View

Already implemented in `bttui.py` as the 5-column side-by-side lifecycle view (Discovery / Connection / Handshake / Data / Errors). Extend by interleaving D-Bus events into appropriate columns, adding click-to-expand for full event detail, and enabling save/export of a specific connection's full flow for bug reports.

Kernel Module Probe

Already implemented in `bt_probe.py` and `bt_probe.rs`. Integrate into bluTruth as the Kernel tab. Add periodic background probing with configurable interval, automatic diff alerting when module parameters change unexpectedly, and timeline integration so probe events appear in the main timeline with SYSFS tag.

Tools Built So Far

File	Language	Purpose	Status
`bttui.py`	Python	Full-featured TUI with 7 tabs, btmon, protocol flow	Complete
`bt_probe.py`	Python	Kernel/firmware probe, SQLite history, diff	Complete
`bt_probe.rs` + `Cargo.toml`	Rust	Same as bt_probe.py, single deployable binary	Complete

The Python and Rust probe tools share the same SQLite schema and JSON format, making their databases fully compatible. A snapshot captured with the Python tool can be diffed against one captured with the Rust binary.

bttui.py tabs: Adapters (HCI adapter table), Devices (with SDP/GATT/L2Ping actions), Scan (Classic and BLE), HCI Monitor (color-coded live btmon with 8-category legend), Protocol Flow (5-column lifecycle view), Stats (adapter stats, link quality), Tools (BlueZ version, D-Bus objects, paired DB, kernel modules).

bt_probe commands:

```bash
python3 bt_probe.py probe            # snapshot, save, print
python3 bt_probe.py watch --interval 60   # periodic snapshots
python3 bt_probe.py history          # list all snapshots
python3 bt_probe.py show <id>        # full detail
python3 bt_probe.py diff <id_a> <id_b> # compare two snapshots
python3 bt_probe.py dump <id>        # raw JSON
python3 bt_probe.py modules          # live module detail
```

D-Bus Object Model: Complete Reference

This is the section that makes bluTruth's D-Bus introspection work. Every interface, every property, every method, every signal that bluetoothd exposes on the system bus is documented here. bluTruth monitors all of it.

The top-level service is `org.bluez`. The root object is `/org/bluez`. Every object under it is reachable via `org.freedesktop.DBus.ObjectManager.GetManagedObjects()` on the root. All property changes are signaled via `org.freedesktop.DBus.Properties.PropertiesChanged`.

Object Hierarchy

```
/org/bluez
  org.bluez.AgentManager1
  org.bluez.ProfileManager1
  org.bluez.HealthManager1

/org/bluez/hci0
  org.bluez.Adapter1
  org.bluez.GattManager1
  org.bluez.LEAdvertisingManager1
  org.bluez.Media1
  org.bluez.NetworkServer1

/org/bluez/hci0/dev_XX_XX_XX_XX_XX_XX
  org.bluez.Device1
  org.bluez.MediaControl1        (if device supports AVRCP)

/org/bluez/hci0/dev_XX_XX_XX_XX_XX_XX/serviceXXXX
  org.bluez.GattService1

/org/bluez/hci0/dev_XX_XX_XX_XX_XX_XX/serviceXXXX/charXXXX
```

```
  org.bluez.GattCharacteristic1

/org/bluez/hci0/dev_XX_XX_XX_XX_XX_XX/serviceXXXX/charXXXX/descXXXX
  org.bluez.GattDescriptor1

/org/bluez/hci0/dev_XX_XX_XX_XX_XX_XX/playerX
  org.bluez.MediaPlayer1

/org/bluez/hci0/dev_XX_XX_XX_XX_XX_XX/fdX
  org.bluez.MediaTransport1       (A2DP or HFP audio transport)
```

`org.bluez.AgentManager1` (on `/org/bluez`)

This interface manages pairing agents. Exactly one agent should be registered at a time. If none is registered, Just Works is used for all pairings requiring user interaction.

Methods:

- `RegisterAgent(agent_path: ObjectPath, capability: string)` — registers a D-Bus object as the pairing agent. Capability is one of: `DisplayOnly`, `DisplayYesNo`, `KeyboardOnly`, `NoInputNoOutput`, `KeyboardDisplay`. The capability tells bluetoothd what input/output the user interface supports, which drives the pairing method selection.
- `UnregisterAgent(agent_path: ObjectPath)` — removes the agent.
- `RequestDefaultAgent(agent_path: ObjectPath)` — promotes an agent to be the default. When multiple agents are registered (unusual), the default is preferred.

The agent object itself must implement `org.bluez.Agent1` at the registered path:

- `RequestPinCode(device: ObjectPath)` → `string` — returns a PIN (for legacy devices)
- `DisplayPinCode(device: ObjectPath, pincode: string)` — show this PIN to the user
- `RequestPasskey(device: ObjectPath)` → `uint32` — returns a passkey
- `DisplayPasskey(device: ObjectPath, passkey: uint32, entered: uint16)` — show passkey with how many digits entered
- `RequestConfirmation(device: ObjectPath, passkey: uint32)` — ask user to confirm passkey matches
- `RequestAuthorization(device: ObjectPath)` — ask user to authorize an incoming connection
- `AuthorizeService(device: ObjectPath, uuid: string)` — ask user to authorize a specific profile service
- `Cancel()` — abort the current request
- `Release()` — agent is being unregistered, clean up

What to watch in bluTruth: Agent method calls show up as D-Bus method calls to your agent object path. If a pairing silently fails, check whether `RequestConfirmation` or `RequestAuthorization` was called and never answered. This is a very common failure mode in headless setups.

`org.bluez.ProfileManager1` (on `/org/bluez`)

For registering custom RFCOMM profiles. An application calls `RegisterProfile` with a UUID, an object path implementing `org.bluez.Profile1`, and options.

Methods:

- `RegisterProfile(profile: ObjectPath, uuid: string, options: dict)` — options include `Name`, `Service` (SDP service name), `Role` (`client` or `server`), `Channel` (RFCOMM channel number), `PSM` (L2CAP PSM for custom L2CAP profiles), `RequireAuthentication` (bool), `RequireAuthorization` (bool), `AutoConnect` (bool), `ServiceRecord` (full XML SDP record).
- `UnregisterProfile(profile: ObjectPath)`

The profile object at the registered path must implement `org.bluez.Profile1`:

- `NewConnection(device: ObjectPath, fd: UnixFD, fd_properties: dict)` — called when a connection is established; the file descriptor is a ready-to-use socket for the RFCOMM channel
- `RequestDisconnection(device: ObjectPath)` — called when the connection should be torn down
- `Release()` — profile is being unregistered

What to watch: If `NewConnection` is never called after a device connects and `ServicesResolved` goes true, the issue is either that the remote device didn't find the service in SDP, or `RequireAuthentication` failed silently.

`org.bluez.Adapter1` (on `/org/bluez/hciN`)

The most-used interface. Controls the physical adapter.

Methods:

- `StartDiscovery()` — begins scanning. On LE, uses active scanning by default. Filter with `SetDiscoveryFilter` first.
- `StopDiscovery()`
- `SetDiscoveryFilter(properties: dict)` — filter properties include `UUIDs` (array of UUIDs to filter by), `RSSI` (minimum signal strength), `Pathloss` (maximum), `Transport` (`auto`, `bredr`, `le`), `DuplicateData` (bool, whether to report repeated advertisements), `Discoverable` (bool), `Pattern` (name prefix).
- `RemoveDevice(device: ObjectPath)` — removes device from the registry and deletes its pairing database entry. This is the safe way to unpair — safer than manually deleting files in `/var/lib/bluetooth/` while the daemon is running.

- `ConnectDevice(properties: dict)` — (BlueZ 5.48+) experimental: connect to an address+type without prior discovery.

Properties (all emit PropertiesChanged):

Property	Type	Description
`Address`	string	Adapter BD address, read-only
`AddressType`	string	`public` or `random`
`Name`	string	System hostname
`Alias`	string	Friendly name, writable
`Class`	uint32	Bluetooth device class
`Powered`	bool	Writable — power on/off the adapter
`Discoverable`	bool	Writable — whether adapter is visible
`DiscoverableTimeout`	uint32	Writable — seconds before discoverable auto-off
`Pairable`	bool	Writable — whether to accept pair requests
`PairableTimeout`	uint32	Writable
`Discovering`	bool	True while scanning
`UUIDs`	string[]	Profiles supported by this adapter
`Modalias`	string	USB modalias (vendor/product info)
`Roles`	string[]	Supported roles: `central`, `peripheral`, `central-peripheral`, `broadcaster`, `observer`

What to watch: `Powered` going false unexpectedly often means rfkill blocked the adapter or a kernel driver failure. `Discovering` going false without you calling `StopDiscovery` means an error condition or another process interfering. Both are important correlation events in bluTruth.

`org.bluez.Device1` (on `/org/bluez/hciN/dev_XX_XX_XX_XX_XX_XX`)

The richest interface — represents a remote device.

Methods:

- `Connect()` — attempts to connect all applicable profiles. This is the high-level connect. bluetoothd determines which profiles to connect based on the device's service class and supported UUIDs.
- `ConnectProfile(uuid: string)` — connect a specific profile by UUID.
- `Disconnect()` — disconnect all profiles.
- `DisconnectProfile(uuid: string)`
- `Pair()` — initiate pairing. Requires an agent to be registered if the device needs user confirmation.
- `CancelPairing()`

Properties:

Property	Type	When Emitted	Description
`Address`	string	Once	BD address
`AddressType`	string	Once	`public` or `random`
`Name`	string	On discovery	Device-reported name
`Alias`	string	Writable	Your name for the device
`Class`	uint32	On discovery	Bluetooth class of device
`Appearance`	uint16	BLE devices	BLE appearance value
`Icon`	string	Derived	Icon name from class/appearance
`Paired`	bool	After pairing	Whether bonded
`Trusted`	bool	Writable	Affects auto-connect behavior
`Blocked`	bool	Writable	Refuses connections if true
`LegacyPairing`	bool	After inquiry	Uses PIN rather than SSP
`RSSI`	int16	During discovery	Signal strength, vanishes after connect
`Connected`	bool	On connect/disconnect	**The most-watched property**
`UUIDs`	string[]	After SDP	Profiles the device supports
`Modalias`	string	From SDP	Vendor/product info
`Adapter`	ObjectPath	Always	Parent adapter
`ManufacturerData`	dict	BLE advert	Manufacturer-specific advertisement data
`ServiceData`	dict	BLE advert	Service-specific advertisement data
`AdvertisingFlags`	bytes	BLE advert	AD flags byte
`AdvertisingData`	dict	BLE advert	Full advertisement data
`ServicesResolved`	bool	After SDP	**Gate for GATT access** — must be true before GATT reads
`WakeAllowed`	bool	Writable	Allow wake from suspend

Property	Type	When Emitted	Description
`TxPower`	int16	BLE advert	Advertised transmit power

The sequence of property changes during a successful BLE connection is the most important pattern to know:

```
Connected: false → true        (HCI Connection Complete)
ServicesResolved: false → true   (all GATT service discovery done, ~500ms-2s after connect)
```

For Classic BR/EDR with audio:

```
Connected: false → true
UUIDs populated / updated
ServicesResolved: false → true
[MediaTransport1 object created on the device path]
MediaTransport1.State: idle → pending → active
```

Any deviation from this sequence is a diagnostic signal. `Connected: true` but `ServicesResolved` never reaching `true` means service discovery is failing or hanging — look at the HCI stream for repeated ATT requests with errors, or SDP requests that timeout.

`org.bluez.GattService1`

Properties:

Property	Type	Description
`UUID`	string	128-bit service UUID
`Device`	ObjectPath	Parent device
`Primary`	bool	True for primary services, false for included
`Includes`	ObjectPath[]	Included services
`Handle`	uint16	ATT handle, useful for correlating with HCI ATT PDUs

`org.bluez.GattCharacteristic1`

Methods:

- `ReadValue(options: dict)` → `bytes` — options can include `offset` (uint16), `mtu` (uint16), `device` (ObjectPath).
- `WriteValue(value: bytes, options: dict)` — options: `offset`, `type` (`command`, `request`, `reliable`), `mtu`, `prepare-authorize`.
- `AcquireWrite(options: dict)` → `(fd: UnixFD, mtu: uint16)` — get a direct socket for writing without D-Bus round-trips. Much faster for streaming characteristics.
- `AcquireNotify(options: dict)` → `(fd: UnixFD, mtu: uint16)` — get a direct socket to receive notifications.
- `StartNotify()` — enable notifications via `PropertiesChanged` on the `Value` property (slower than `AcquireNotify` but simpler).
- `StopNotify()`

Properties:

Property	Type	Description
`UUID`	string	128-bit characteristic UUID
`Service`	ObjectPath	Parent service
`Value`	bytes	Current cached value, updated by notifications
`WriteAcquired`	bool	Whether a write socket is currently acquired
`NotifyAcquired`	bool	Whether a notify socket is currently acquired
`Notifying`	bool	Whether notifications are active
`Flags`	string[]	`broadcast`, `read`, `write-without-response`, `write`, `notify`, `indicate`, `authenticated-signed-writes`, `extended-properties`, `reliable-write`, `writable-auxiliaries`, `encrypt-read`, `encrypt-write`, `encrypt-authenticated-read`, `encrypt-authenticated-write`, `secure-read`, `secure-write`, `authorize`
`Handle`	uint16	ATT handle
`MTU`	uint16	Current ATT MTU for this characteristic

`org.bluez.MediaTransport1`

This is the audio transport object created when bluetoothd successfully establishes an A2DP or HFP/HSP transport. It lives at a path like `/org/bluez/hci0/dev_XX_XX_XX_XX_XX_XX/fdN`.

Methods:

- `Acquire()` → `(fd: UnixFD, read_mtu: uint16, write_mtu: uint16)` — called by the audio server to take ownership of the transport. Returns a socket for audio data.
- `TryAcquire()` → same — non-blocking; returns an error if the transport isn't ready.
- `Release()` — called by audio server when it's done.

Properties:

Property	Type	Description
`Device`	ObjectPath	The device this transport belongs to
`UUID`	string	Profile UUID (A2DP source/sink, HFP, HSP)
`Codec`	byte	Codec ID (A2DP: 0x00=SBC, 0x02=AAC, 0xFF=vendor)
`Configuration`	bytes	Codec configuration blob
`State`	string	**`idle`** → **`pending`** → **`active`**
`Delay`	uint16	Transport delay in 1/10th ms, writable
`Volume`	uint16	Volume 0–127, writable if supported

`State` transitions are the key audio debug signal:

- `idle` — transport created but not acquired by audio server
- `pending` — audio server has requested acquisition, waiting for device confirmation
- `active` — audio is flowing

If the transport sits in `pending` for more than a second and never reaches `active`, the device failed to confirm the transport setup. Check the HCI stream for the A2DP stream start procedure (AVDTP SET_CONFIGURATION → OPEN → START) and look for where it failed.

`org.bluez.LEAdvertisingManager1` (on `/org/bluez/hciN`)

For applications that want the local adapter to advertise as a BLE peripheral.

Methods:

- `RegisterAdvertisement(advertisement: ObjectPath, options: dict)`
- `UnregisterAdvertisement(advertisement: ObjectPath)`

Properties:

- `ActiveInstances` (byte) — how many advertisement instances are currently active
- `SupportedInstances` (byte) — how many the controller supports
- `SupportedIncludes` (string[]) — what data can be included: `tx-power`, `appearance`, `local-name`
- `SupportedSecondaryChannels` (string[]) — for Bluetooth 5 extended advertising

`org.bluez.GattManager1` (on `/org/bluez/hciN`)

For registering a GATT server application (peripheral role).

Methods:

- `RegisterApplication(application: ObjectPath, options: dict)` — the application object must implement `org.freedesktop.DBus.ObjectManager` and expose child objects implementing `GattService1`, `GattCharacteristic1`, `GattDescriptor1`.
- `UnregisterApplication(application: ObjectPath)`

Complete D-Bus Monitoring Command Reference

```bash
```

```
# Snapshot: everything bluetoothd knows right now
busctl call org.bluez / \
  org.freedesktop.DBus.ObjectManager GetManagedObjects \
  | python3 -c "
import sys, dbus, json
# Or pipe the busctl output through a formatter
"

# Real-time: all signals and property changes from bluez
busctl monitor org.bluez

# Real-time: just property changes (less noise)
dbus-monitor --system \
  "type='signal',sender='org.bluez',interface='org.freedesktop.DBus.Properties',member='PropertiesChanged'"

# Real-time: interface add/remove (devices appearing/disappearing)
dbus-monitor --system \
  "type='signal',sender='org.bluez',interface='org.freedesktop.DBus.ObjectManager'"

# Real-time: specific device properties only
dbus-monitor --system \
  "type='signal',sender='org.bluez',path='/org/bluez/hci0/dev_AA_BB_CC_DD_EE_FF'"

# Introspect any object
busctl introspect org.bluez /org/bluez/hci0
busctl introspect org.bluez /org/bluez/hci0/dev_AA_BB_CC_DD_EE_FF

# Get a specific property
busctl get-property org.bluez /org/bluez/hci0/dev_AA_BB_CC_DD_EE_FF \
  org.bluez.Device1 Connected

# Set a property (e.g., power adapter on)
busctl set-property org.bluez /org/bluez/hci0 \
  org.bluez.Adapter1 Powered b true

# Trigger a method call (e.g., connect a device)
busctl call org.bluez /org/bluez/hci0/dev_AA_BB_CC_DD_EE_FF \
  org.bluez.Device1 Connect

# Remove a device (the safe unpair)
busctl call org.bluez /org/bluez/hci0 \
  org.bluez.Adapter1 RemoveDevice o \
  /org/bluez/hci0/dev_AA_BB_CC_DD_EE_FF
```

```
# Watch a transport's state change during audio connection
dbus-monitor --system \
  "type='signal',sender='org.bluez',interface='org.freedesktop.DBus.Properties',member='PropertiesChanged',arg0='org.bluez.M
```

Troubleshooting Decision Trees

This section provides systematic diagnostic paths for the most common Bluetooth failure classes. Use in combination with bluTruth's correlated timeline.

Failure: Device Won't Pair

```
Does bluetoothctl show the device during scan?
  NO  → Check adapter is powered and discovering
        Check device is in pairing mode
        Check rfkill isn't blocking: rfkill list
        Check btmon for any inquiry/scan traffic at all
        If no HCI traffic: kernel/hardware layer problem → Layer 0/1
  YES ↓

Does pairing attempt start at all in btmon?
  NO  → Check D-Bus: is there an agent registered?
        Run: busctl call org.bluez /org/bluez/hci0/dev_XX... org.bluez.Device1 Pair
        Check dbus-monitor for AccessDenied errors
  YES ↓

Does btmon show LMP/SMP messages?
  NO  → Connection is failing before pairing reaches crypto layer
        Look for: HCI Connection Complete with non-zero status
        Status 0x04 = page timeout (device out of range or not responding)
        Status 0x08 = connection timeout
        Status 0x16 = connection terminated by local host
  YES ↓

Does pairing complete at the HCI level (Encryption Change event)?
  NO  → Authentication failure at crypto layer
        Check: device and host capability mismatch (IO capability)
        Check: agent was called but didn't respond (busctl monitor during pairing)
        Check: device rejected the passkey or confirmation
  YES ↓
```

```
Does bluetoothd log show "Storage" or "Storing" messages? (run bluetoothd -n -d)
  NO  → Pairing DB write failure — check /var/lib/bluetooth permissions
  YES ↓

Does the Paired property appear on the device D-Bus object?
  NO  → bluetoothd internal state machine didn't complete — try restart
  YES → Pairing succeeded. If it doesn't reconnect, see reconnect tree below.
```

Failure: Device Pairs But Won't Connect

```
Does btmon show a connection attempt after calling Connect()?
  NO  → bluetoothd isn't sending the HCI command
        Check: device isn't Blocked (busctl get-property ... Blocked)
        Check: adapter Pairable and Powered
        Try: busctl call org.bluez /org/bluez/hci0/dev_XX org.bluez.Device1 Connect
        Check dbus-monitor for error replies to Connect()
  YES ↓

Does the HCI Connection Complete event show status = 0x00?
  NO  → See HCI status codes table. Most common:
        0x05 = authentication failure (keys don't match — wipe and re-pair)
        0x08 = connection timeout (device out of range)
        0x13 = remote terminated (device refused)
        0x16 = local terminated (bluetoothd gave up)
  YES ↓

Does btmon show profile connection attempts (SDP query or RFCOMM/A2DP setup)?
  NO  → Connection succeeded but bluetoothd didn't try to connect profiles
        Check UUIDs property — does it list the expected profiles?
        If ServicesResolved is false, SDP may have failed — check for SDP PDUs in btmon
  YES ↓

Do profile connections complete successfully?
  NO  → Profile-specific failure. See audio tree (if A2DP/HFP) or check RFCOMM error
  YES ↓

Does the Connected property on the D-Bus object go true?
  NO  → bluetoothd didn't update its state despite HCI success — bluetoothd bug
        Collect: bluetoothd -n -d output for this sequence
  YES → Connection succeeded. If audio doesn't work, see audio tree.
```

Failure: Audio Connects But No Sound (A2DP)

```
Is the MediaTransport1 object created on the device path?
  NO  → A2DP profile didn't connect. See profile connection tree above.
      busctl tree org.bluez | grep fd
  YES ↓

Does the MediaTransport1.State reach 'active'?
  NO, stays 'idle'  → Audio server (PipeWire/PulseAudio) never called Acquire()
                      Check: is the BT sink showing in audio server?
                      pw-cli list-objects | grep bluez
                      pactl list cards | grep bluez
  NO, stuck 'pending' → Audio server called Acquire() but device didn't confirm
                        Look in btmon for AVDTP START command and device response
                        If device sent REJECT: codec configuration mismatch
  YES ↓

Is audio data flowing through the transport socket?
  Check: btmon for ACL data packets with A2DP header (0x00 L2CAP PSM for A2DP)
  If ACL data present but no audio: codec decode issue (wrong configuration bytes)
  If no ACL data: audio server isn't writing to the transport fd

Common A2DP codec configuration failures:
  SBC: Both sides support it but configuration bytes don't match
      (sampling rate, channel mode, block length, subband count, allocation method, bitpool)
  AAC: Device claims to support AAC but rejects the configuration
      Try forcing SBC: comment out AAC in /etc/pipewire/media-session.d/bluez-monitor.conf
  aptX/LDAC: Vendor codecs require device-side support AND correct Linux fw/plugin
```

Failure: HFP Microphone Doesn't Work / Call Audio Broken

```
HFP is fundamentally different from A2DP — it uses SCO (Synchronous Connection Oriented)
rather than ACL for audio. SCO is a circuit-switched channel that bypasses the normal
L2CAP/ACL data path. This makes it much harder to debug.

Does btmon show HFP profile setup (RFCOMM channel + AT command exchange)?
  NO  → HFP profile not connecting. See profile tree.
  YES ↓

Does btmon show the HFP AT command exchange completing?
  Look for: AT+BIA, AT+BRSF, AT+CIND, AT+CMER, AT+CHLD
  If exchange incomplete: one side doesn't support the required features
  YES ↓
```

```
Does btmon show SCO connection setup attempt?
  Look for: "Synchronous Connection Complete" event
  NO  → Neither side initiated SCO.
        PipeWire/PA may not have connected HFP module.
        Check: journalctl -u pipewire | grep -i hfp
        Check: pactl list cards (should show HFP profile)
  YES ↓

Does SCO Synchronous Connection Complete show status = 0x00?
  NO  → Common failure: eSCO negotiation failed, fell back to SCO, then that failed
        Status 0x1F = unspecified error (very common with cheap BT chips)
        Try: echo 1 > /sys/module/bluetooth/parameters/disable_esco
        (Forces plain SCO instead of eSCO — lower quality but more compatible)
  YES ↓

Is audio flowing but with quality issues?
  Underruns/crackle: SCO buffer size mismatch in audio server
  One-way only (can't hear caller): Check if CVOICE codec negotiation specified
  Echo: Missing echo cancellation in HFP path (PA/PW has EC module)
```

Failure: Device Randomly Disconnects

```
Check btmon for the disconnect event:
  "Disconnection Complete" — what is the reason code?

  0x08 = Connection timeout    → Device moved out of range, or device-side power saving
  0x13 = Remote user terminated → Device deliberately disconnected (app on device side)
  0x14 = Remote low resources  → Device ran out of buffers (typically a device firmware bug)
  0x15 = Remote power off      → Device battery died or user turned it off
  0x16 = Connection terminated by local host → bluetoothd or kernel chose to disconnect
  0x3E = Connection failed to establish → LE-specific, parameters incompatible
  0x22 = LL response timeout   → LE link layer timeout (too many missed connection events)

  0x16 (local host) is the most diagnostic — it means Linux chose to disconnect.
  Look backwards in the correlated timeline:
    - Any error in the HCI stream before it?
    - Any bluetoothd log message about profile failure?
    - Any kernel trace event?
    - Any suspend/resume activity (check dmesg)?

  0x08 (timeout) recurring → Check USB autosuspend:
    cat /sys/bus/usb/devices/*/power/autosuspend_delay_ms
```

```
echo -1 > /sys/bus/usb/devices/X-X.X/power/autosuspend_delay_ms
Or add to /etc/udev/rules.d/50-usb-bluetooth.rules:
ACTION=="add", SUBSYSTEM=="usb", ATTRS{idVendor}=="XXXX", ATTRS{idProduct}=="XXXX", \
ATTR{power/autosuspend_delay_ms}="-1"
```

---

bluTruth Implementation Roadmap

Phase 0 — Foundation (Complete)

The diagnostic foundation exists:

- `bttui.py` — 7-tab TUI: Adapters, Devices, Scan, HCI Monitor (color-coded, 8-category classification), Protocol Flow (5-column lifecycle view), Stats, Tools
- `bt_probe.py` — kernel/firmware probe with SQLite history and diff
- `bt_probe.rs` + `Cargo.toml` — identical Rust binary, compatible database

These tools prove the concept. Phase 1 integrates them into a unified platform.

Phase 1 — Unified Collection Engine

Goal: Single process that simultaneously ingests all 7 diagnostic streams and writes them to a unified SQLite timeline.

Tasks:

1. **HCI broadcaster refactor** — current bttui.py runs btmon as a subprocess and parses its output. Refactor to a proper broadcaster class that parses HCI lines into structured dicts with fields: `ts_mono_us`, `direction` (TX/RX), `type` (CMD/EVT/ACL/SCO), `opcode_or_event`, `status`, `handle`, `payload_hex`, `summary_text`, `stage`, `severity`. Fan out to multiple consumers (TUI tab, SQLite writer, HTTP SSE stream, btsnoop file writer).
2. **D-Bus monitor module** — new `dbus_monitor.py` using `dbus-python` (or `dbus-next` for async). Subscribes to all `PropertiesChanged`, `InterfacesAdded`, `InterfacesRemoved` on `org.bluez`. Emits events in the same normalized format. This is the core new feature of Phase 1.
3. **bluetoothd log capture** — subprocess launcher that optionally manages the daemon and captures stderr directly. Fallback to `journalctl -f -u bluetooth` for systems using systemd.
4. **Kernel trace collector** — optional module (requires root + debugfs) that tails `/sys/kernel/debug/tracing/trace_pipe`. Lightweight filter: only emit events with `bluetooth` in the function name.
5. **SQLite writer** — single-writer thread (important for SQLite performance) that drains a queue and inserts events. Uses WAL mode. Target: >1000 inserts/second sustained.

6. **Correlation pass** — runs asynchronously over recent events. Links HCI events to D-Bus events that occurred within ±100ms. Groups linked events under a shared `group_id`.

Deliverable: `blucollect.py` — a daemon that can run headless and write to `~/.blutruth.db`. The TUI and HTTP server connect to the database.

Phase 2 — D-Bus Introspection Tab

Goal: The "100% D-Bus introspection" view requested. Live tree of everything bluetoothd knows.

Layout:

```
┌─ Object Tree ──────────────────────────┬─ Properties ──────────────────────────────────┐
│ /org/bluez                             │ org.bluez.Device1                             │
│   AgentManager1                        │ ──────────────────────────                    │
│   hci0  [Powered: ON]                  │ Connected:      true          ← 14:32:01      │
│     dev_AA_BB_CC  [Connected]          │ Paired:         true                          │
│       Device1                          │ ServicesResolved: true        ← 14:32:03      │
│       MediaTransport1 [active]         │ UUIDs:          [110b, 110e, 1108]            │
│         GattService 1800               │ RSSI:           -67                           │
│           Char 2A00 [read]             ├─ Recent Property Changes ─────────────────────┤
│           Char 2A01 [read]             │ 14:32:01.234  Connected: false → true         │
│         GattService 1801               │ 14:32:01.891  ServicesResolved: false → true  │
│                                        │ 14:32:02.103  Transport State: idle → active  │
└────────────────────────────────────────┴───────────────────────────────────────────────┘
```

Left pane auto-updates as objects appear and disappear. Right pane shows all current property values with the timestamp of the last change, plus a rolling log of changes below.

Clicking a characteristic in the left pane opens a detail view: UUID, flags, current value in hex + decoded, option to send ReadValue or StartNotify.

Implementation: Python `dbus-next` (async) + Textual. The object tree is rebuilt from `GetManagedObjects()` on startup, then updated incrementally via `InterfacesAdded`/`InterfacesRemoved` signals. No polling.

Phase 3 — Controls Panel

Goal: All stack management operations in one place, with pre/post snapshots for every action.

Operations and their implementations:

```
Adapter Operations:
  Power On/Off        → busctl set-property org.bluez /org/bluez/hci0
                        org.bluez.Adapter1 Powered b true/false
  Discoverable On/Off → same, Discoverable property
  Reset (soft)        → hciconfig hci0 down && sleep 1 && hciconfig hci0 up
  Reset (hard)        → rmmod btusb && sleep 2 && modprobe btusb
  Full stack bounce   → systemctl stop bluetooth
                        rmmod bnep rfcomm btusb bluetooth
                        sleep 3
                        modprobe bluetooth && modprobe btusb
                        systemctl start bluetooth

Device Operations:
  Connect/Disconnect  → org.bluez.Device1.Connect() / Disconnect()
  Pair/Unpair         → org.bluez.Device1.Pair() / Adapter1.RemoveDevice()
  Trust/Untrust       → set Trusted property
  Block/Unblock       → set Blocked property
  Wipe pairing entry  → Adapter1.RemoveDevice() [does both DB and memory]

Daemon Operations:
  Restart (systemd)   → systemctl restart bluetooth
  Restart (debug)     → systemctl stop bluetooth
                        bluetoothd -n -d &  (captured in log stream)
  Stop/Start          → systemctl stop/start bluetooth

Pairing DB Operations:
  View raw entry      → cat /var/lib/bluetooth/<adapter>/<device>/info
  Export all          → cp -r /var/lib/bluetooth/ <backup_path>
  Wipe single device  → Adapter1.RemoveDevice() is preferred
                        Manual: stop daemon, rm -rf /var/lib/bluetooth/<adapter>/<device>/
                        restart daemon
  Wipe adapter DB     → stop daemon, rm -rf /var/lib/bluetooth/<adapter>/
                        restart daemon
```

Every operation captures a `bt_probe` snapshot before and after, allowing diff to show what changed.

Phase 4 — Correlated Timeline and Web UI

Goal: The unified timeline view — the core value proposition of bluTruth — implemented in both TUI and browser.

TUI Timeline Tab:

- Scrolling event log showing all sources simultaneously

- Each line: `[timestamp] [SOURCE] [DEVICE] [STAGE] summary`
- Color by severity: normal/warn/error/suspicious
- Lines from different sources that are correlated (same logical event) are visually grouped with a connecting bracket
- Filters: source, device, stage, severity, time range
- Search: grep-style across summaries
- Zoom: expand any correlated group to see full event detail

HTTP + React SPA:

- `GET /stream` — SSE endpoint streaming normalized events as JSON
- `GET /state` — current adapter/device state
- `GET /events?since=&limit=&source=&device=` — historical query
- `POST /control` — JSON body with action and parameters
- React frontend with the same timeline view as the TUI, rendered in browser
- Shareable URLs for a specific time range
- Export: download a time range as JSON or as a btsnoop file

Technology stack:

- Python `aiohttp` for HTTP server (integrates cleanly with the async D-Bus and btmon collection)
- React + Vite for the SPA, served as static files from the same process
- SSE (Server-Sent Events) for real-time push — simpler than WebSocket, sufficient for this use case

Phase 5 — Anomaly Detection

Goal: Auto-flag known bad patterns without requiring the user to know what to look for.

Pattern library:

Pattern Name	HCI Signature	D-Bus Signature	Severity
KNOB attack	Encryption key length < 7 bytes in LMP Encryption Key Size Response	—	CRITICAL
BIAS attack	Authentication procedure skipped on reconnect	—	HIGH
SSP downgrade	IO capability exchange results in Just Works when both devices have screens	—	HIGH
PIN fallback	Legacy pairing used when SSP should be available	WARN	MEDIUM
Repeated pair fail	3+ pairing failures from same address in 60s	—	MEDIUM
Role switch spam	Unexpected role switch requests from device	—	LOW
eSCO downgrade	SCO negotiation falls back from eSCO to SCO	—	LOW
Fast repeated disconnect	Device disconnects within 5s of connecting, 3+ times	Connected cycling	MEDIUM
Codec downgrade	A2DP falls back to SBC from higher codec	Transport config	LOW
SDP flood	Unusual number of SDP requests from device	—	LOW

Anomaly events appear in a dedicated tab and are flagged in the timeline with a warning icon. Each anomaly links to documentation explaining what it means and what to do.

Blind Spots

Areas of the stack where current tools have no or very limited visibility:

Controller internals — the controller firmware is entirely opaque. You see only what it chooses to tell you via HCI events. Controller-side bugs, internal scheduling decisions, and RF-level failures are invisible unless the controller reports them as error events — which poorly implemented firmware often does not.

SCO/eSCO audio path — once an SCO connection is established, the audio data flows through a separate kernel path. You can see the connection setup in btmon but not the audio quality, jitter, or packet error rate on the SCO link unless the controller sends SCO link statistics events (rare in consumer hardware).

bluetoothd internals without debug logging — without the `-d` flag, bluetoothd's internal decision-making is a black box. You see inputs and outputs but not the logic connecting them. This is one of the strongest arguments for having bluTruth optionally manage the daemon with debug logging enabled.

PipeWire and PulseAudio BT plugin internals — the audio server's Bluetooth plugin has its own state machine managing the audio transport. Its internal decisions (why it rejected a codec, why it dropped a buffer) are not visible from the BT stack side.

Cross-device pairing state — you typically only have visibility into the Linux host side. The device's internal state, if accessible at all, requires vendor tools. This asymmetry makes certain failure classes very hard to diagnose definitively.

Commands and Programs Reference

BlueZ and HCI tools

```bash
bluetoothctl                    # Interactive CLI for all BT operations
btmon                           # HCI monitor — capture all HCI traffic
btmon -w file.btsnoop           # Save to Wireshark btsnoop format
btmgmt                          # Direct mgmt API access, lower level than bluetoothctl
hciconfig                       # Adapter configuration (older, partially deprecated)
hciconfig hci0 -a               # Full adapter info
hciconfig hci0 version          # HCI/LMP version and firmware string
hciconfig hci0 up / down        # Enable/disable adapter
hcidump                         # Raw HCI dump (older, btmon preferred)
sdptool                         # SDP service record operations
sdptool browse <addr>           # Browse services on a remote device
l2ping <addr>                   # L2CAP ping test
gatttool                        # GATT client (older, replaced by bluetoothctl gatt)
hcitool                         # Scan, info, connections (mostly deprecated)
```

D-Bus tools

```bash
dbus-monitor --system "sender=org.bluez"   # Watch all org.bluez traffic
busctl tree org.bluez                      # Object hierarchy
busctl introspect org.bluez /org/bluez/hci0  # Adapter interface
busctl monitor org.bluez                   # Live property changes
dbus-send --system --print-reply \
  --dest=org.bluez /org/bluez/hci0 \
  org.freedesktop.DBus.Properties.GetAll \
  string:org.bluez.Adapter1                # Get all adapter properties programmatically
```

Kernel and sysfs

```bash
lsmod | grep -i bluetooth            # Loaded BT modules
modinfo bluetooth                    # Core module detail
modinfo btusb                        # USB driver detail
ls /sys/module/bluetooth/parameters/   # Runtime parameters
ls /sys/class/bluetooth/             # Detected adapters
ls /sys/kernel/debug/bluetooth/        # debugfs detail (root required)
cat /proc/net/bluetooth/l2cap        # Open L2CAP sockets
dmesg | grep -iE '(bluetooth|btusb)'   # Kernel BT messages
```

systemd

```bash
systemctl status bluetooth
systemctl restart bluetooth
journalctl -u bluetooth -f
journalctl -u bluetooth --since "1 hour ago"
systemctl stop bluetooth && bluetoothd -n -d   # Manual debug launch
```

Wireshark

Wireshark fully decodes btsnoop captures from btmon using the BTatt, BTl2cap, BThci_cmd, BThci_evt, and BThci_acl dissectors:

```bash
btmon -w /tmp/capture.btsnoop
wireshark /tmp/capture.btsnoop
```

Kernel tracing

```bash
echo 1 > /sys/kernel/debug/tracing/events/bluetooth/enable
cat /sys/kernel/debug/tracing/trace_pipe
echo 0 > /sys/kernel/debug/tracing/events/bluetooth/enable
```

Adjacent tools with no direct BT stack visibility

`rfkill` — blocks/unblocks BT at the RF kill switch hardware level, completely separate from the BT stack.

`iwconfig` and `iw` — WiFi tools, relevant only for BT/WiFi coexistence investigation. `strace -p $(pidof bluetoothd)` — syscall trace of the daemon; heavy but gives socket-level visibility. `lsof -p $(pidof bluetoothd)` — open file descriptors including sockets and the pairing DB files.

Links for More Info

Official documentation and source

- BlueZ official site and documentation: http://www.bluez.org/
- BlueZ GitHub repository: https://github.com/bluez/bluez
- BlueZ D-Bus API documentation: https://git.kernel.org/pub/scm/bluetooth/bluez.git/tree/doc
- Linux kernel Bluetooth subsystem documentation: https://www.kernel.org/doc/html/latest/networking/bluetooth.html
- Linux kernel Bluetooth source (net/bluetooth/): https://elixir.bootlin.com/linux/latest/source/net/bluetooth
- btusb driver source: https://elixir.bootlin.com/linux/latest/source/drivers/bluetooth/btusb.c
- bluetooth.ko core source: https://elixir.bootlin.com/linux/latest/source/net/bluetooth/hci_core.c
- BlueZ mgmt API specification: https://git.kernel.org/pub/scm/bluetooth/bluez.git/tree/doc/mgmt-api.txt

Bluetooth specifications

- Bluetooth Core Specification (all versions): https://www.bluetooth.com/specifications/specs/core54-html/
- Bluetooth Assigned Numbers (opcodes, UUIDs, company IDs): https://www.bluetooth.com/specifications/assigned-numbers/
- Bluetooth SIG specifications archive: https://www.bluetooth.com/specifications/specs/

Deep technical references

- Wireshark Bluetooth dissectors source: https://gitlab.com/wireshark/wireshark/-/tree/master/epan/dissectors
- An Introduction to Bluetooth programming (Albert Huang, MIT): https://people.csail.mit.edu/albert/bluez-intro/
- Bluetooth stack in Linux — LWN article: https://lwn.net/Articles/564036/
- BlueZ internals — Linux kernel mailing list: https://lkml.org/

Security research

- KNOB Attack — key negotiation of Bluetooth: https://knobattack.com/
- BIAS Attack — Bluetooth impersonation attacks: https://francozappa.github.io/about-bias/

- BlueBorne vulnerabilities (Armis): https://www.armis.com/research/blueborne/
- InternalBlue — Bluetooth firmware reverse engineering toolkit: https://github.com/seemoo-lab/internalblue
- SWEYNTOOTH vulnerabilities in BLE: https://asset-group.github.io/disclosures/sweyntooth/

Alternative stacks and tools

- btstack by BlueKitchen (embedded/userspace stack): https://github.com/bluekitchen/btstack
- Wireshark Bluetooth capture analysis: https://wiki.wireshark.org/Bluetooth
- scapy Bluetooth layers: https://scapy.readthedocs.io/en/latest/layers/bluetooth.html

PipeWire and audio integration

- PipeWire Bluetooth documentation: https://gitlab.freedesktop.org/pipewire/pipewire/-/wikis/Bluetooth
- PipeWire spa-bluez5 plugin source: https://gitlab.freedesktop.org/pipewire/pipewire/-/tree/master/spa/plugins/bluez5

Firmware resources

- Linux firmware repository: https://git.kernel.org/pub/scm/linux/kernel/git/firmware/linux-firmware.git
- Realtek BT firmware notes: https://git.kernel.org/pub/scm/linux/kernel/git/firmware/linux-firmware.git/tree/rtl_bt
- Intel BT firmware notes: https://git.kernel.org/pub/scm/linux/kernel/git/firmware/linux-firmware.git/tree/intel

---

This document is the living knowledge base for the bluTruth project. Update this file as the architecture evolves and new diagnostic capabilities are added. Last updated: 2026-02-23